

Enci/ VENTAS

Inventario Técnico del Repositorio

ANEXO A

Proyecto Enci / Enci Ventas

Aplicación de Automatización de Fuerza de Ventas

Equipo de Desarrollo: Grupo PLM - Universidad Tecnológica Metropolitana

Líder: Joshua Flores

Integrantes: Falon Berrios, Carolaine Palacios y Eliseo Poblete

Versión 1.0

Julio 2026

Tabla de Contenidos

1. Introducción y propósito
2. Identificación del proyecto y responsables
3. Alcance del inventario técnico
4. Estructura principal del repositorio
5. Backend API FastAPI
6. Frontend web React + Vite
7. Aplicación vendedor y componente móvil Flutter
8. Documentación técnica versionada
9. Configuración, infraestructura y despliegue
10. Seguridad, autorización y control de acceso
11. Pruebas, QA y validación técnica
12. Trazabilidad hacia la entrega documental
13. Observaciones de traspaso técnico
14. Criterios de aceptación del anexo

1. Introducción y propósito

Este anexo presenta el inventario técnico del repositorio oficial asociado al proyecto Enci / Enci Ventas. Su objetivo es ordenar los componentes que componen el sistema, identificar las carpetas y archivos relevantes, describir su función dentro de la solución y dejar una base clara para auditoría, mantención, soporte, despliegue y traspaso técnico.

El documento está orientado a revisión técnica y recepción por parte del cliente. La información se estructura desde una perspectiva de operación real: qué artefactos existen, qué capa del sistema representan, qué uso tienen dentro del cierre y qué consideraciones deben mantenerse durante la continuidad del proyecto.

Naturaleza del anexo
Este documento no reemplaza el repositorio ni los manuales de instalación, despliegue, usuario, API, seguridad o soporte. Su función es consolidar el mapa técnico del proyecto y relacionar los artefactos principales con el alcance de cierre.

1.1 Objetivos específicos

- Identificar la estructura del monorepo y sus carpetas principales.
- Describir los componentes backend, frontend, documentación, configuración y pruebas.
- Relacionar cada artefacto con su utilidad dentro del MVP web avanzado.
- Dejar constancia de los componentes asociados a RAG, seguridad, readiness, roles y QA.
- Distinguir los artefactos presentes en el repositorio de componentes relacionados que se entregan por fuera del monorepo, como la app nativa Flutter parcialmente realizada.
- Facilitar el traspaso técnico a operación, mantención o una fase posterior.

2. Identificación del proyecto y responsables

Campo	Detalle
Proyecto	Enci / Enci Ventas
Tipo de sistema	MiniCRM comercial y plataforma de automatización de fuerza de ventas
Repositorio técnico	Enci, monorepo oficial del proyecto
Cliente / representante receptor	Max
Equipo responsable	Grupo PLM - Universidad Tecnológica Metropolitana
Líder del equipo	Joshua Flores
Integrantes	Falon Berrios, Carolaine Palacios y Eliseo Poblete
Versión del documento	1.0
Fecha de emisión	Julio 2026

3. Alcance del inventario técnico

El inventario cubre los artefactos técnicos necesarios para entender el estado del sistema entregado y su relación con el cierre del proyecto. Se considera como base el MVP web avanzado, compuesto por backend FastAPI, frontend React/Vite, autenticación Firebase, persistencia Firestore, documentación técnica, pruebas y módulos comerciales.

También se incorporan componentes complementarios que deben entenderse con alcance específico: IA RAG con DeepSeek como módulo parcialmente entregado y aplicación móvil Flutter nativa Android como entregable externo al repositorio, parcialmente realizada conforme a lo conversado y validado con el representante del cliente.

Ámbito	Estado dentro del inventario	Criterio de lectura
MVP web comercial	Incluido	Inventariado como componente principal de la entrega técnica.
Backend FastAPI	Incluido	API, seguridad, roles, reglas de negocio, endpoints y pruebas.
Frontend React/Vite	Incluido	Interfaz web SPA, rutas protegidas, pantallas comerciales y módulos de gestión.
Firestore y Firebase	Incluido	Persistencia, autenticación, reglas deny-all y validación por backend.
IA RAG	Incluido como parcial	Inventariado por sus endpoints, servicios, configuración, proveedor y límites de alcance.
App vendedor PWA	Incluida si existe en monorepo	Referenciada como artefacto web/móvil instalable asociado al repositorio.
App nativa Flutter	Relacionado externo	No se evalúa por presencia en GitHub; se considera entregable parcial externo conforme a evidencia y acuerdo de cierre.
ERP, SAP, tiendas móviles, MDM	Fuera de inventario base	Requieren fase posterior, acuerdo adicional o definición técnica nueva.

4. Estructura principal del repositorio

La estructura principal del repositorio responde a una organización de monorepo, separando backend, frontend, documentación, configuración, herramientas y artefactos de validación. Esta división permite aislar responsabilidades, simplificar despliegues y mantener trazabilidad técnica entre código, documentación y evidencias de QA.

Ruta	Descripción	Uso en cierre
Backend/	API FastAPI, modelos, servicios, routers, configuración, autenticación y pruebas automatizadas.	Backend operativo del MVP web avanzado.

Ruta	Descripción	Uso en cierre
frontend/	Aplicación web React + Vite para CRM comercial, dashboards y módulos de operación.	Interfaz principal del equipo comercial.
vendedor-app/	PWA vendedor React/Vite asociada a experiencia móvil instalable desde navegador.	Base móvil web asociada al proyecto; no reemplaza la app Flutter externa.
docs/	Documentación técnica, QA, API, arquitectura, sprints, hardening y evidencias de cierre.	Soporte documental de traspaso técnico.
tools/	Scripts auxiliares para smoke test, auditoría, verificación y generación de documentación.	Validación, automatización y soporte de entrega.
firebase.json	Configuración base de Firebase para el proyecto.	Referencia de infraestructura Firebase.
firestore.rules	Reglas de seguridad Firestore.	Control de bloqueo de acceso directo desde cliente.
README.md	Resumen técnico y operativo del proyecto.	Punto de entrada para instalación, alcance y estado general.

4.1 Lectura técnica del monorepo

La carpeta Backend concentra la lógica de negocio y la autorización efectiva. El frontend organiza la experiencia visual, pero no actúa como fuente de seguridad. La documentación del repositorio permite verificar decisiones de arquitectura, endpoints, setup, QA, hardening y procedimientos de salida. Las herramientas complementarias apoyan la validación local y la generación de evidencias.

5. Backend API FastAPI

El backend corresponde al núcleo operativo del sistema. Su responsabilidad es exponer la API REST, validar tokens Firebase, aplicar roles, proteger operaciones por propiedad, conectarse con Firestore, ejecutar reglas comerciales, entregar readiness y servir documentación técnica controlada.

Ruta	Función técnica
Backend/app/main.py	Inicializa FastAPI, registra routers, CORS, rate limit, límite de tamaño de request, handlers de error, headers de seguridad, health, readiness y /me.

Ruta	Función técnica
Backend/app/core/config.py	Centraliza variables de entorno, validaciones de seguridad, CORS, Firebase, DeepSeek, Chroma, GCS y límites del módulo RAG.
Backend/app/core/auth.py	Valida Authorization Bearer, verifica token Firebase, consulta usuario en Firestore, normaliza rol, valida estado activo y acceso por plataforma.
Backend/app/core/readiness.py	Construye reporte no sensible de condiciones mínimas para UAT/go-live.
Backend/app/core/rate_limit.py	Aplica límite por IP, ventana de control y restricción de tamaño de payload.
Backend/app/core/headers.py	Aplica headers dinámicos de caché y soporte de hardening HTTP.
Backend/app/core/errors.py	Normaliza respuestas de error HTTP, validación, errores Google API y errores no controlados.
Backend/app/api/*.py	Define routers por dominio: autenticación, clientes, comercial, dashboard, competencia, RAG, usuarios y documentación.
Backend/app/models/*.py	Contiene schemas Pydantic, validaciones, normalización de datos y control de campos de entrada.
Backend/app/services/*.py	Agrupar reglas de negocio y acceso a Firebase, Firestore, dashboard, clientes, comercial, RAG y LLM.
Backend/tests/*.py	Pruebas automatizadas para permisos, readiness, seguridad, RAG, usuarios y flujos comerciales.

5.1 Routers funcionales

Router	Responsabilidad principal	Criterio de aceptación técnica
auth	Login, registro y sincronización de usuario autenticado.	Debe exigir token Firebase válido y persistir perfil operativo.
clientes	CRUD y baja lógica de clientes.	Debe aplicar visibilidad por vendedor y acceso superior por supervisor/admin.
comercial	Interacciones, oportunidades y propuestas.	Debe mantener relación entre cliente, oportunidad, propuesta y usuario autorizado.
dashboard	Métricas, forecast, embudo, equipo y alertas.	Debe retornar datos agregados según rol y alcance de visibilidad.
competencia	Registro y análisis de información competitiva.	Debe operar bajo token y reglas de negocio del dominio.

Router	Responsabilidad principal	Criterio de aceptación técnica
rag	Chat documental, conversaciones y administración de documentos RAG.	Debe separar acceso vendedor/supervisor/admin y proteger carga documental para admin.
users	Gestión de usuarios, roles, accesos y estados.	Debe restringirse a perfiles autorizados y registrar cambios críticos.
docs	Swagger/OpenAPI bajo condiciones controladas.	Debe estar disponible para pruebas y restringido según configuración productiva.

5.2 Servicios de negocio

Los servicios backend separan la lógica de negocio de los routers. Esta separación permite mantener control sobre validaciones, composición de respuestas, consultas Firestore, cálculo de métricas, procesamiento RAG y manejo de proveedor LLM. Para mantención futura, cualquier cambio funcional debe realizarse preferentemente en servicios y modelos antes que duplicarse en routers o frontend.

6. Frontend web React + Vite

El frontend web corresponde a la interfaz principal del MVP comercial. Está construido como SPA React/Vite y consume la API protegida mediante token Firebase. Su responsabilidad es ofrecer navegación, pantallas, formularios, visualización de datos y experiencia por rol, sin reemplazar las reglas de autorización del backend.

Ruta	Función
frontend/src/App.jsx	Define rutas principales del CRM, protección de vistas y navegación general.
frontend/src/auth/	Contiene AuthProvider, control de sesión, roles, rutas protegidas y contexto de usuario.
frontend/src/services/api.js	Cliente API autenticado; adjunta token Bearer y gestiona errores de sesión.
frontend/src/pages/	Pantallas de login, dashboard, clientes, detalle, oportunidades, propuestas, proyecciones, equipo, inteligencia, RAG, competencia y configuración.
frontend/src/components/	Componentes reutilizables como AppShell, formularios, selectores, estados visuales, modales y elementos RAG.
frontend/src/utls/	Utilidades para clientes, forecast, analytics, exportación, normalización y manejo de errores API.

Ruta	Función
frontend/.env.example	Referencia de variables públicas VITE_* para desarrollo y despliegue.
frontend/package.json	Dependencias, scripts de lint, build y ejecución local.

6.1 Pantallas y módulos web relacionados

Módulo	Descripción operativa
Login	Inicio de sesión con Google SSO mediante Firebase Auth.
Dashboard	KPIs, métricas comerciales, forecast, embudo, alertas y visión por rol.
Clientes	Listado, búsqueda, filtros, creación, edición, detalle y baja lógica.
Interacciones	Registro de llamadas, correos, reuniones, visitas y otros contactos comerciales.
Oportunidades	Pipeline, etapas, filtros, detalle e historial relacionado.
Propuestas	Creación vinculada a oportunidad, estados y cálculo server-side.
Equipo de ventas	Vista consolidada de rendimiento, matriz por etapa, zonas y vendedores.
Asistente Enci	Interfaz para consultas RAG y continuidad de conversaciones.
Competencia	Registro y análisis de datos competitivos según alcance comercial.
Configuración	Usuarios, tema, idioma, rangos, zonas y preferencias según rol.

7. Aplicación vendedor y componente móvil Flutter

El proyecto contempla dos elementos móviles que deben distinguirse para evitar ambigüedades de traspaso: una PWA vendedor asociada al repositorio y una app nativa Flutter parcialmente realizada fuera del monorepo. Ambas tienen naturaleza distinta y no deben evaluarse bajo el mismo criterio de inventario.

7.1 App vendedor presente en repositorio

Ruta	Función
vendedor-app/src/App.jsx	Define rutas principales de la PWA vendedor.
vendedor-app/src/pages/	Contiene vistas de inicio, pipeline, cotizaciones, proyección, configuración y Enci Chat.
vendedor-app/src/services/api.js	Cliente API de la PWA para comunicación con backend.
vendedor-app/src/services/cotizacionesService.js	Soporte funcional para cotizaciones y flujos asociados.
vendedor-app/api/[...path].js	Proxy Vercel hacia backend FastAPI publicado.

Ruta	Función
vendedor-app/public/manifest.webmanifest	Configuración de instalación PWA.
vendedor-app/package.json	Scripts, dependencias y configuración de build de la PWA.

7.2 App nativa Flutter parcialmente realizada

La app nativa Flutter Android se considera un componente relacionado de entrega, pero no alojado dentro del monorepo oficial revisado. Su ausencia en GitHub no constituye, por sí sola, ausencia de entregable. Su criterio de aceptación corresponde al estado parcial acordado con el representante del cliente y a las evidencias de demostración, pruebas y revisión final disponibles en el cierre.

Ámbito	Criterio técnico de inventario
Naturaleza	Aplicación móvil nativa Android desarrollada en Flutter, relacionada con el módulo vendedor.
Ubicación	Componente externo al repositorio GitHub del monorepo Enci.
Estado de entrega	Parcialmente realizada / en integración final, según acuerdo de cierre.
Funciones base	Login Google/Firebase, navegación principal, nueva cotización, pipeline, proyección/forecast, configuración y sesión, según demostración y alcance parcial.
Integración backend	No se exige cobertura total de endpoints FastAPI dentro del alcance parcial.
Distribución	APK Android o evidencia funcional equivalente; no incluye publicación en tiendas, firma productiva, MDM ni soporte post-entrega.
Cierre	Solicitudes futuras sobre Flutter se consideran mejora, Fase 2 o nuevo acuerdo.

8. Documentación técnica versionada

La carpeta docs cumple un rol central dentro del traspaso, porque conserva arquitectura, setup, endpoints, QA, seguridad, cierre, troubleshooting y planificación histórica. Esta documentación permite reconstruir decisiones del proyecto y comprobar la relación entre código y criterios de aceptación.

Documento	Uso técnico
docs/architecture.md	Arquitectura general, stack, roles, monorepo y alcance MVP.
docs/setup.md	Instalación, configuración local y pasos base para levantar el ambiente.
docs/cierre-proyecto.md	Resumen de cierre, alcance y criterios de salida.
docs/api/auth.md	Contrato y uso de endpoints de autenticación.

Documento	Uso técnico
docs/api/clientes.md	Contrato de API para gestión de clientes.
docs/api/comercial.md	Interacciones, oportunidades y propuestas.
docs/api/dashboard.md	Endpoints y criterios de dashboard comercial.
docs/api/rag.md	Endpoints, permisos, DeepSeek, Chroma, carga documental y límites RAG.
docs/api/users.md	Gestión de usuarios, roles, estado activo y accesos.
docs/qa/epic-05-uat-go-live.md	Plan UAT, readiness, smoke test, defectos y go-live.
docs/qa/hardening-seguridad.md	Hardening, riesgos, mitigaciones y comandos de verificación.
docs/troubleshooting/api-errors.md	Guía de errores API y acciones correctivas.

9. Configuración, infraestructura y despliegue

El sistema depende de configuración segura y separación de ambientes. Las variables sensibles no deben versionarse, y los dominios autorizados deben configurarse tanto en Firebase Auth como en CORS backend. El despliegue previsto separa frontend, backend y servicios administrados.

Elemento	Responsabilidad	Consideración de cierre
Firebase Auth	Autenticación Google SSO y emisión de ID Token.	Requiere dominios autorizados del ambiente final.
Firebase Admin SDK	Validación backend de tokens Firebase.	Credenciales deben permanecer fuera del repositorio.
Cloud Firestore	Persistencia de usuarios, clientes y flujo comercial.	Acceso directo de cliente bloqueado por reglas deny-all.
Cloud Run	Destino previsto para backend FastAPI.	Requiere variables productivas y prueba de readiness.
Vercel	Destino previsto para frontend web y PWA vendedor.	Variables VITE_* y base URL pública deben estar correctas.
DeepSeek	Proveedor LLM del módulo RAG.	API key solo en backend; no debe exponerse en frontend.
Chroma	Índice local persistente para recuperación documental.	Debe respaldarse si contiene corpus relevante.
GCS Bucket	Almacenamiento productivo previsto para documentos RAG.	Depende de configuración de ambiente.

9.1 Variables de entorno críticas

Variable / grupo	Uso	Sensibilidad
APP_ENV, APP_NAME, APP_VERSION	Identificación del backend y comportamiento por ambiente.	No sensible.
CORS_ORIGINS, CORS_ORIGIN_REGEX	Control de orígenes permitidos.	Configuración crítica.
FIREBASE_PROJECT_ID	Proyecto Firebase objetivo.	Crítica, no secreta.
GOOGLE_APPLICATION_CREDENTIALS	Ruta de credencial backend Firebase Admin.	Sensible indirecta.
FIREBASE_SERVICE_ACCOUNT_JSON	Credencial de servicio en formato variable.	Sensible.
VITE_FIREBASE_*	Configuración pública frontend Firebase.	Pública, pero debe coincidir con ambiente.
DEEPSEEK_API_KEY	Llave de proveedor RAG.	Secreta, backend exclusivamente.
GCS_BUCKET_DOCUMENTS	Bucket productivo de documentos RAG.	Crítica según ambiente.
CHROMA_PERSIST_DIR	Directorio de índice vectorial.	Operativa.
RAG_MAX_UPLOAD_BYTES	Límite de carga documental.	Operativa / seguridad.

10. Seguridad, autorización y control de acceso

La seguridad del sistema se concentra en backend. El frontend puede ocultar acciones, pero la decisión obligatoria de acceso se realiza mediante token, estado de usuario, rol, plataforma y propiedad de registros. Esta regla debe mantenerse en cualquier continuidad del proyecto.

Control	Ubicación	Propósito
Bearer token Firebase	Backend/app/core/auth.py	Validar identidad antes de ejecutar lógica protegida.
Roles admin/supervisor/vendedor	Backend/app/core/auth.py y servicios	Restringir operaciones según perfil.
Usuario activo	Firestore + backend	Bloquear usuarios inactivos aun con token válido.
Acceso por plataforma	Backend appMovil/webApp	Controlar acceso entre web y app móvil.
Firestore deny-all	firestore.rules	Evitar lectura/escritura directa desde cliente.
Rate limit	Middleware backend	Mitigar ráfagas básicas y abuso del MVP.

Control	Ubicación	Propósito
Request size limit	Middleware backend	Bloquear payloads sobredimensionados.
CORS restrictivo	Config backend	Limitar orígenes permitidos.
Headers de seguridad	Middleware HTTP	Reducir superficie ante framing, sniffing y políticas inseguras.
Validaciones Pydantic	Backend/app/models	Rechazar campos inesperados y datos inválidos.

10.1 Regla de autorización obligatoria

Criterio de seguridad

Toda operación sensible debe validarse en backend. No se debe considerar suficiente un bloqueo visual del frontend, una ruta oculta o una condición de interfaz. La autorización efectiva se define mediante API, token, rol, estado activo y propiedad del recurso.

11. Pruebas, QA y validación técnica

El repositorio mantiene pruebas y procedimientos de validación para confirmar que los flujos críticos no dependan solo de revisión manual. La continuidad del proyecto debe mantener estos comandos como barrera mínima antes de merge, despliegue o entrega UAT.

Validación	Comando / ubicación	Resultado esperado
Backend tests	cd Backend; uv run pytest	Suite completa en verde.
Frontend lint	cd frontend; npm run lint	Sin errores bloqueantes.
Frontend build	cd frontend; npm run build	Build generado sin fallos.
PWA vendedor lint	cd vendedor-app; npm run lint	Sin errores bloqueantes.
PWA vendedor build	cd vendedor-app; npm run build	Build generado correctamente.
Readiness	GET /readiness	Estado ready u observaciones no sensibles del ambiente.
Healthcheck	GET /health	Respuesta status=ok.
Swagger controlado	/docs y /openapi.json según ambiente	Disponible para pruebas y restringido en producción.
Smoke test	tools/ o procedimiento documentado	Validación post despliegue de servicios principales.

12. Trazabilidad hacia la entrega documental

El inventario técnico se relaciona con el resto de la documentación de cierre. Cada documento especializado toma parte de estos artefactos y los desarrolla desde una perspectiva distinta: operación, instalación, API, seguridad, usuario, QA, respaldo o exclusiones.

Documento relacionado	Relación con este inventario
DOC-02 Resumen ejecutivo técnico	Resume arquitectura, estado del MVP, stack, endpoints y reglas de autorización.
DOC-03 Manual de instalación local	Usa rutas, dependencias y comandos identificados en el repositorio.
DOC-04 Guía de variables de entorno	Detalla variables backend, frontend, Firebase y RAG.
DOC-05 Manual de despliegue	Aplica a los componentes backend, frontend, PWA y servicios de infraestructura.
DOC-08 Manual de roles y permisos	Desarrolla la autorización identificada en auth.py, modelos y servicios.
DOC-11 Documentación API / Swagger	Relaciona routers, endpoints y contratos REST.
DOC-14 Informe QA y validación	Usa tests, lint, build, readiness y smoke test como evidencia.
DOC-15 Seguridad y hardening	Amplía controles de auth, CORS, headers, payload, Firestore y errores.
DOC-16 Documentación RAG	Detalla el módulo RAG, DeepSeek, Chroma, documentos y conversaciones.
DOC-17 App vendedor	Distingue PWA del repositorio y app Flutter parcial externa.
DOC-19 Registro de exclusiones	Controla componentes fuera de alcance o sujetos a fase posterior.

13. Observaciones de traspaso técnico

- La autorización real reside en backend; el frontend no debe considerarse control de seguridad suficiente.
- Firestore debe permanecer bloqueado para lectura y escritura directa desde clientes finales.
- Todo ambiente desplegado requiere coherencia entre dominios Firebase Auth, CORS backend y variables públicas VITE_*.
- Las credenciales Firebase Admin y DEEPSEEK_API_KEY son secretos backend y no deben versionarse.
- El módulo RAG depende de proveedor externo, corpus documental, índice Chroma y configuración de almacenamiento; su entrega se considera parcial según alcance de cierre.

- La app Flutter nativa Android es un entregable parcial externo al monorepo; no debe confundirse con la PWA vendedor presente en el repositorio.
- Toda modificación futura sobre Flutter, RAG avanzado, publicación en tiendas, ERP/SAP, SLA productivo, MDM o soporte post-entrega debe tratarse como nueva fase o nuevo acuerdo.
- El README y la documentación docs/ deben mantenerse actualizados cada vez que cambie un endpoint, flujo, variable o condición de despliegue.

14. Criterios de aceptación del anexo

Este anexo se considera aceptable si permite identificar los componentes principales del repositorio, comprender su función, relacionarlos con el cierre documental y diferenciar los elementos del MVP web respecto de entregables parciales o externos.

Código	Criterio de aceptación	Resultado esperado
CA-A01	El documento identifica backend, frontend, documentación, configuración, seguridad y pruebas.	El lector puede ubicar los artefactos críticos sin revisar todo el repositorio.
CA-A02	Se distingue la PWA vendedor presente en repo de la app Flutter externa parcial.	No se interpreta erróneamente la ausencia de Flutter en GitHub como inexistencia del entregable parcial.
CA-A03	Se documenta la dependencia de Firebase, Firestore, FastAPI, React/Vite, DeepSeek y Chroma.	El traspaso técnico reconoce dependencias obligatorias.
CA-A04	Se explicita que la autorización efectiva está en backend.	No se delega seguridad a navegación frontend.
CA-A05	Se relaciona el inventario con documentos de cierre posteriores.	La entrega documental queda trazable y ordenada.
CA-A06	Se evita prometer continuidad no pactada sobre RAG, Flutter, ERP, tiendas móviles o soporte.	El alcance queda protegido para cierre y futuras fases.

Cierre del anexo
El inventario técnico queda establecido como mapa de referencia para auditoría, mantención y traspaso del proyecto Enci / Enci Ventas. Cualquier diferencia entre repositorio, ambiente desplegado y entregables externos debe resolverse contra evidencia técnica vigente y criterios de cierre aprobados.